

AI ASSSITED CODING

LAB 18.4

NAME : SD.Nayaz

ENROLL.NO:2403A52076

BATCH:04

TASK-01:

Connect to a Public API• Instructions:• Use Python (or Node.js/JavaScript) to connect to a publicAPI (e.g., OpenWeatherMap or JSON Placeholder).• Send a simple GET request to retrieve data.• Display the response in a readable format (pretty JSON)

PROMPT:

Write a simple program that connects to a public API (such as OpenWeatherMap, JSON Placeholder, or another free API), sends a GET request, and displays the data in a readable (pretty JSON) format.

CODE:

```
task18.4.1.py X task18.4.2.py task18.4.3.py task18.4.4.py task18.4.5.py
lab18.4 > task18.4.1.py > fetch_json
1  import sys
2  import json
3  from urllib.request import urlopen, Request
4  from urllib.error import URLError, HTTPError
5  #!/usr/bin/env python3
6  """
7  Fetch JSON from JSONPlaceholder and print pretty JSON.
8  Usage:
9  python task18.4.1.py          # fetch /posts
10 python task18.4.1.py posts    # same as above
11 python task18.4.1.py posts 1  # fetch /posts/1
12 python task18.4.1.py users    # fetch /users
13 """
14 BASE = "https://jsonplaceholder.typicode.com"
15
16 def fetch_json(path="/posts"):
17     url = BASE + path
18     req = Request(url, headers={"User-Agent": "python-urllib/3"})
19     try:
20         with urlopen(req, timeout=10) as resp:
21             raw = resp.read()
22             return json.loads(raw.decode("utf-8"))
23     except HTTPError as e:
24         print(f"HTTP error: {e.code} {e.reason}")
25         sys.exit(1)
26     except URLError as e:
27         print(f"URL error: {e.reason}")
28         sys.exit(1)
29     except Exception as e:
30         print(f"Unexpected error: {e}")
31         sys.exit(1)
32
33 def main():
34     # build path from optional CLI args
35     if len(sys.argv) == 1:
36         path = "/posts"
37     elif len(sys.argv) == 2:
38         path = "/" + sys.argv[1].lstrip("/")
39     else:
40         # e.g. posts 1 -> /posts/1
41         path = "/" + sys.argv[1].lstrip("/") + "/" + sys.argv[2].lstrip("/")
42
43     data = fetch_json(path)
44     pretty = json.dumps(data, ensure_ascii=False, indent=2)
45     print(pretty)
46
47 if __name__ == "__main__":
48     main()
```

OUTPUT:

```
PS C:\Users\ranch\OneDrive\Desktop\ai> & C:\Users\ranch\AppData\Local\Programs\Python\Python312\python.exe c:\Users\ranch\OneDrive\Desktop\ai\lab18.4\task18.4.1.py
[{"userId": 1,
  "id": 1,
  "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",
  "body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas totam\nnostrum rerum est autem sunt rem eveniet architecto"
},
{"userId": 1,
  "id": 2,
  "title": "qui est esse",
  "body": "est rerum tempore vitae\nsequi sint nihil reprehenderit dolor beatae ea dolores neque\nfugiat blanditiis voluptate porro vel nihil molestiae ut reiciendis\nqui aperiam non debitis possimus qui neque nisi nulla"
},
{"userId": 1,
  "id": 3,
  "title": "ea molestias quasi exercitationem repellat qui ipsa sit aut",
  "body": "et iusto sed quo iure\nvoluptatem occaecati omnis eligendi aut ad\nvoluptatem doloribus vel accusantium quis pariatur\nmolestiae porro eius odio et labore et velit aut"
},
{"userId": 1,
  "id": 4,
  "title": "eum et est occaecati",
  "body": "ullam et saepe reiciendis voluptatem adipisci\nsit amet autem assumenda provident rerum culpa\nquis hic commodi nesciunt rem tenetur doloremque ipsam iure\nquis sunt voluptatem rerum illo velit"
},
{"userId": 1,
  "id": 5,
  "title": "nesciunt quas odio",
  "body": "repudiandae veniam quaerat sunt sed\nnihil aut fugiat sit autem sed est\nvoluptatem omnis possimus esse voluptatibus quis\nvest aut tenetur dolor neque"
},
{"userId": 1,
  "id": 6,
  "title": "dolorem eum magni eos aperiam quia",
  "body": "ut aspernatur corporis harum nihil quis provident sequi\nmollitia nobis aliquid molestiae\nperspiciatis et ea nemo ab reprehenderit accusantium quas\nvoluptate dolores velit et dolor enim eum mollitia"
},
{"userId": 1,
  "id": 7,
  "title": "magnam facillis autem",
  "body": "dolore placeat quibusdam ea quo vitae\nmagni quis enim qui quis quo nemo aut saepe\nquidem repellat excepturi ut quia\nsunt ut sequi eos ea sed quas"
},
{"userId": 1,
  "id": 8,
  "title": "dolorem dolore est ipsam",
  "body": "dignissimos aperiam dolorem qui eum\nfacilis quibusdam animi sint suscipit qui sint possimus cum\nquaerat magni maiores excepturi\nipsam ut commodi dolor voluptatum modi aut vitae"
}]
```

OBSERVATION:

AI has created a script that fetches sample data from the JSONPlaceholder API. I can choose what data to pull by giving different arguments, like posts or posts/1. If the request fails, my program clearly tells me what went wrong. Whatever data I get is printed in a neat, readable JSON format. Overall, it helps me quickly view API data in a simple way.

TASK-02:

Add Error Handling for Invalid API Calls

- Instructions:
 - o Modify your code from Task 1 to handle errors.
 - o Include try/except (Python) or try/catch (JavaScript) blocks.
 - o Handle cases like:
 - Invalid URL
 - Network timeout
 - Wrong API key (if required)
 - o Print user-friendly error messages.

PROMPT:

Write a program that gets data from a public API. Use try/except (or try/catch) to handle errors and show clear messages. If it works, display the data neatly.

CODE:

```
task18.4.1.py task18.4.2.py X task18.4.3.py task18.4.4.py task18.4.5.py
lab18.4 > task18.4.2.py > main
1 import os
2 import sys
3 import json
4 import requests
5
6 # /c:/Users/ramch/OneDrive/Desktop/ai/lab18.4/task18.4.2.py
7
8 API_URL = os.getenv("API_URL", "https://api.example.com/data") # replace with real URL
9 API_KEY = os.getenv("API_KEY", "") # set your API key in env var or here
10
11 def main():
12     headers = {"Authorization": f"Bearer {API_KEY}" if API_KEY else ""}
13     try:
14         resp = requests.get(API_URL, headers=headers, timeout=5)
15         # Raise for HTTP errors (4xx, 5xx)
16         resp.raise_for_status()
17         # If API returns JSON, print it similar to Task 1
18         try:
19             data = resp.json()
20             print(json.dumps(data, indent=2))
21         except ValueError:
22             # Not JSON, print raw text
23             print(resp.text)
24     except requests.exceptions.InvalidURL:
25         print("Error: Could not connect to API. Check your API key or network connection.")
26         sys.exit(1)
27     except requests.exceptions.Timeout:
28         print("Error: Could not connect to API. Check your API key or network connection.")
29         sys.exit(1)
30     except requests.exceptions.ConnectionError:
31         print("Error: Could not connect to API. Check your API key or network connection.")
32         sys.exit(1)
33     except requests.exceptions.HTTPError as he:
34         # Handle wrong API key (401) and other HTTP errors uniformly
35         status = getattr(he.response, "status_code", None)
36         if status == 401:
37             print("Error: Could not connect to API. Check your API key or network connection.")
38         else:
39             print("Error: Could not connect to API. Check your API key or network connection.")
40             sys.exit(1)
41     except requests.exceptions.RequestException:
42         print("Error: Could not connect to API. Check your API key or network connection.")
43         sys.exit(1)
44
45 if __name__ == "__main__":
46     main()
```

OUTPUT:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS QUERY RESULTS
PS C:\Users\ramch\OneDrive\Desktop\ai> & C:/Users/ramch/AppData/Local/Programs/Python/Python312/python.exe c:/Users/ramch/OneDrive/Desktop/ai/lab18.4/task18.4.2.py
• {
  "userId": 1,
  "id": 1,
  "title": "delectus aut autem",
  "completed": false
}
PS C:\Users\ramch\OneDrive\Desktop\ai>
```

OBSERVATION:

This script uses AI-powered tools to make calling an API simple and reliable. It sends a request to a chosen URL and automatically checks for any errors. If the API returns JSON, the AI-friendly code formats it neatly for easy reading. If the response isn't JSON, it still shows the raw result clearly. Overall, it's a clean and smart way to fetch and display API data.

TASK-03:

Extract and Display Specific Data • Instructions: 1. From the API response (e.g., weather API), extract specific fields (temperature, humidity, description). 2. Display them in a user-friendly format (not raw JSON)

PROMPT:

Get data from a public API (like a weather API). Pick out only a few details (for example: temperature, humidity, description). Show these details in a **clear and easy-to-read** way — not raw JSON.

CODE:

```
lab18.4 > task18.4.3.py > format_weather
1  import os
2  import sys
3  import json
4  import requests
5
6  # /c:/Users/ranch/OneDrive/Desktop/ai/lab18.4/task18.4.3.py
7  # Extract specific fields (temperature, humidity, description) from a weather API response
8  # Usage:
9  # - python task18.4.3.py # runs example with embedded sample JSON
10 # - python task18.4.3.py --file resp.json
11 # - python task18.4.3.py --fetch London (requires OPENWEATHER_API_KEY env var)
12
13
14 def format_weather(data):
15     # Safe extraction with sensible defaults
16     city = data.get("name") or data.get("city") or "Unknown"
17     main = data.get("main", {})
18     weather_list = data.get("weather") or []
19     weather0 = weather_list[0] if weather_list else {}
20     # Temperature handling: OpenWeatherMap returns Kelvin by default.
21     temp = main.get("temp")
22     if temp is None:
23         temp_str = "N/A"
24     else:
25         # If temp looks like Kelvin (> 100) convert to Celsius, else assume already Celsius
26         try:
27             t = float(temp)
28             if t > 100:
29                 t = t - 273.15
30             temp_str = f"{round(t)}°C"
31         except Exception:
32             temp_str = str(temp)
33     humidity = main.get("humidity")
34     humidity_str = f"{humidity}%" if humidity is not None else "N/A"
35     description = weather0.get("description") or weather0.get("main") or "N/A"
36     # Print in user-friendly format
37     print(f"City: {city}")
38     print(f"Temperature: {temp_str}")
39     print(f"Humidity: {humidity_str}")
40     print(f"Weather: {description.capitalize()}")
41
42 def load_json_file(path):
43     with open(path, "r", encoding="utf-8") as f:
44         return json.load(f)
45
46 def fetch_openweathermap(city):
47     try:
48         pass
```

```

lab18.4 > task18.4.3.py > format_weather
46 def fetch_openweather(city):
47     pass
48 except Exception:
49     print("requests library required for fetching from API. Install with: pip install requests", file=sys.stderr)
50     return None
51 key = os.environ.get("OPENWEATHER_API_KEY")
52 if not key:
53     print("Set OPENWEATHER_API_KEY environment variable to fetch live data.", file=sys.stderr)
54     return None
55 url = "https://api.openweathermap.org/data/2.5/weather"
56 params = {"q": city, "appid": key}
57 r = requests.get(url, params=params, timeout=10)
58 r.raise_for_status()
59 return r.json()
60
61
62 SAMPLE_RESPONSE = {
63     "name": "London",
64     "main": {"temp": 291.15, "humidity": 60},
65     "weather": [{"description": "clear sky"}]
66 }
67
68 def main(argv):
69     if len(argv) == 0:
70         data = SAMPLE_RESPONSE
71     elif argv[0] == "--file" and len(argv) > 1:
72         data = load_json_file(argv[1])
73     elif argv[0] == "--fetch" and len(argv) > 1:
74         data = fetch_openweather(argv[1])
75     if data is None:
76         return
77     else:
78         print("Usage:", file=sys.stderr)
79         print(" python task18.4.3.py          # example", file=sys.stderr)
80         print(" python task18.4.3.py --file resp.json", file=sys.stderr)
81         print(" python task18.4.3.py --fetch London # needs OPENWEATHER_API_KEY", file=sys.stderr)
82     return
83     format_weather(data)
84
85 if __name__ == "__main__":
86     main(sys.argv[1:])

```

OUTPUT:

```

PS C:\Users\ranch\OneDrive\Desktop\ai> & C:\Users\ranch\AppData\Local\Programs\Python\Python312\python.exe c:\Users\ranch\OneDrive\Desktop\ai\lab18.4\task18.4.3.py
• City: London
• Temperature: 18°C
• Humidity: 60%
Weather: Clear sky
PS C:\Users\ranch\OneDrive\Desktop\ai>

```

OBSERVATION:

This script uses clean, AI-guided logic to extract useful weather details from any JSON response. It smartly reads temperature, humidity, and descriptions—even converting Kelvin to Celsius automatically. The code can load data from a file, fetch live weather, or use a built-in sample, making it very flexible. If something is missing, it still produces friendly, readable output without breaking. Overall, it's a well-structured, AI-inspired tool for displaying weather information clearly and reliably.

TASK-04:

Build a Function with Parameters• Instructions:• Write a function that accepts a parameter (e.g., city name for weather API).• The function should call the API dynamically based on user input.• Include error handling if the city is invalid.

PROMPT:

Make a function that takes a **parameter** (like a city name). Use that parameter to call an **API** and get data (for example, weather info for that city). Add **error handling** to show a friendly message if the city name is invalid

CODE:

```
task18.4.1.py task18.4.2.py task18.4.3.py task18.4.4.py X task18.4.5.py
lab18.4 > task18.4.4.py > _
1 import os
2 import requests
3
4 # task18.4.4.py
5 # Simple dynamic weather lookup using OpenWeatherMap API.
6 # Set your API key as environment variable OPENWEATHER_API_KEY or replace 'YOUR_API_KEY_HERE'.
7
8
9 API_KEY = os.getenv("OPENWEATHER_API_KEY") or "YOUR_API_KEY_HERE"
10
11 def get_weather(city: str, api_key: str = API_KEY):
12     """
13     Query OpenWeatherMap for the given city and print a simple summary.
14     Handles invalid city names and basic network/API errors.
15     """
16     if not api_key or api_key == "YOUR_API_KEY_HERE":
17         raise RuntimeError("No API key provided. Set OPENWEATHER_API_KEY or replace YOUR_API_KEY_HERE.")
18
19     url = "https://api.openweathermap.org/data/2.5/weather"
20     params = {"q": city, "appid": api_key, "units": "metric"}
21
22     try:
23         resp = requests.get(url, params=params, timeout=10)
24     except requests.exceptions.RequestException as exc:
25         print(f"Error: Network problem or request failed: {exc}")
26         return
27
28     if resp.status_code == 404:
29         print("Error: City not found. Please enter a valid city.")
30         return
31     if resp.status_code != 200:
32         # Try to extract message from response body
33         try:
34             msg = resp.json().get("message", resp.text)
35         except ValueError:
36             msg = resp.text
37         print(f"Error: API returned status {resp.status_code}: {msg}")
38         return
39
40     try:
41         data = resp.json()
42     except ValueError:
43         print("Error: Failed to parse API response.")
44         return
45
46     name = data.get("name", city)
47     main = data.get("main", {})
48     weather_list = data.get("weather", [])
```



```

lab18.4 > task18.4.4.py > ...
11 def get_weather(city: str, api_key: str = API_KEY):
46     name = data.get("name", city)
47     main = data.get("main", {})
48     weather_list = data.get("weather", [])
49     temp = main.get("temp")
50     humidity = main.get("humidity")
51     description = weather_list[0].get("description") if weather_list else "N/A"
52
53     print(f"City: {name}")
54     if temp is not None:
55         print(f"Temperature: {round(temp)}°C")
56     else:
57         print("Temperature: N/A")
58     if humidity is not None:
59         print(f"Humidity: {humidity}%")
60     else:
61         print("Humidity: N/A")
62     print(f"Weather: {description.title()}")
63
64 if __name__ == "__main__":
65     city_input = input("Input: ").strip()
66     if city_input:
67         get_weather(city_input)
68     else:
69         print("No city entered.")

```

OUTPUT:

```

PS C:\Users\ramch\OneDrive\Desktop\ai> PS C:\Users\ramch\OneDrive\Desktop\ai> & C:/Users/ramch/AppData/Local/Programs/Python/Python312/python.exe c:/Users/ramch/OneDrive/Desktop/ai/lab18.4/task18.4.4.py
>> Input: London
>> City: London
>> Temperature: 14°C
>> Humidity: 72%
>> Weather: Broken Clouds
>>

```

OBSERVATION:

This program asks the user for a city name and fetches live weather data from the OpenWeatherMap API. It checks for network issues, missing data, or invalid city names and shows helpful error messages. If the request succeeds, it neatly prints the city's temperature, humidity, and weather description. Overall, it provides a quick and user-friendly way to look up weather information from the terminal.

TASK-05:

Store API Results Locally • Instructions: • Extend your function from Task 4. • Save the extracted API results into a local file (results.json or results.txt). • Each new request should append results without overwriting old ones.

PROMPT:

Update your function to **save API results** to a local file (like results.json or results.txt). Make sure each new result **adds to the file** instead of replacing the old data.

CODE:

```
lab18.4 > task18.4.5.py > ...
1  import os
2  import sys
3  import json
4  import argparse
5  from typing import Optional
6  import requests
7
8  """
9  File: task18.4.5.py
10 Fetch weather for a city (OpenWeatherMap), print formatted output and store results
11 in a local JSON file (results.json). Each call appends a new entry into the JSON array.
12
13 Usage:
14   python task18.4.5.py London
15   or set environment variable OPENWEATHER_API_KEY and run without args to be prompted.
16 """
17
18
19 try:
20     pass
21 except Exception as e:
22     sys.exit("Missing dependency 'requests'. Install with: pip install requests")
23
24 API_URL = "https://api.openweathermap.org/data/2.5/weather"
25 RESULTS_FILE = "results.json"
26
27
28 def fetch_weather(city: str, api_key: str) -> Optional[dict]:
29     params = {"q": city, "appid": api_key, "units": "metric"}
30     resp = requests.get(API_URL, params=params, timeout=10)
31     if resp.status_code != 200:
32         print(f"Error fetching data for '{city}': {resp.status_code} {resp.text}")
```


lab18.4 > task18.4.5.py > ...

```
28 def fetch_weather(city: str, api_key: str) -> Optional[dict]:
29     return None
30
31 data = resp.json()
32 # Extract required fields
33 name = data.get("name", city)
34 main = data.get("main", {})
35 weather_list = data.get("weather", [])
36 temp = main.get("temp")
37 humidity = main.get("humidity")
38 weather_desc = weather_list[0].get("description") if weather_list else ""
39 entry = {
40     "city": name,
41     "temp": int(round(temp)) if temp is not None else None,
42     "humidity": humidity,
43     "weather": weather_desc.title() if isinstance(weather_desc, str) else weather_desc,
44 }
45 return entry
46
47
48
49
50
51 def append_result(entry: dict, filename: str = RESULTS_FILE) -> None:
52     # Load existing array or start a new one
53     results = []
54     if os.path.exists(filename):
55         try:
56             with open(filename, "r", encoding="utf-8") as f:
57                 results = json.load(f)
58                 if not isinstance(results, list):
59                     results = []
60         except Exception:
61             # If file is corrupt or not JSON, overwrite with a fresh list
62             results = []
63     results.append(entry)
```

task18.4.5.py X

lab18.4 > task18.4.5.py > ...

```
51 def append_result(entry: dict, filename: str = RESULTS_FILE) -> None:
64     with open(filename, "w", encoding="utf-8") as f:
65         json.dump(results, f, indent=2, ensure_ascii=False)
66
67
68 def print_formatted(entry: dict) -> None:
69     city = entry.get("city", "Unknown")
70     temp = entry.get("temp", "N/A")
71     humidity = entry.get("humidity", "N/A")
72     weather = entry.get("weather", "")
73     print(f"City: {city}")
74     print(f"Temperature: {temp}°C")
75     print(f"Humidity: {humidity}%")
76     print(f"Weather: {weather}")
77
78
79 def main():
80     parser = argparse.ArgumentParser(description="Fetch weather and store results locally.")
81     parser.add_argument("cities", nargs="+", help="City name(s) to fetch (e.g. London).")
82     parser.add_argument("--api-key", "-k", help="OpenWeatherMap API key (or set OPENWEATHER_API_KEY).")
83     parser.add_argument("--file", "-f", default=RESULTS_FILE, help="Results file (default: results.json).")
84     args = parser.parse_args()
85
86     api_key = args.api_key or os.getenv("OPENWEATHER_API_KEY")
87     if not api_key:
88         api_key = input("Enter OpenWeatherMap API key: ").strip()
89     if not api_key:
90         sys.exit("API key is required.")
91
92     cities = args.cities
93     if not cities:
94         city = input("Enter city name: ").strip()
```

```

lab18.4 > task18.4.5.py > ...
79  def main():
93      if not cities:
94          city = input("Enter city name: ").strip()
95          if not city:
96              sys.exit("No city provided.")
97          cities = [city]
98
99      for city in cities:
100         entry = fetch_weather(city, api_key)
101         if entry:
102             print_formatted(entry)
103             append_result(entry, filename=args.file)
104
105
106  if __name__ == "__main__":
107      main()

```

OUTPUT:

```

PS C:\Users\ramch\OneDrive\Desktop\ai> & C:/Users/ramch/AppData/Local/Programs/Python/Python312/python.exe c:/Users/ramch/OneDrive/Desktop/ai/
lab18.4/task18.4.5.py
Enter OpenWeatherMap API key: 818d5d28d56a6b76906457db5dc1b06d
Enter city name: hyderabad
City: Hyderabad
Temperature: 20°C
Humidity: 73%
Weather: Mist
PS C:\Users\ramch\OneDrive\Desktop\ai>

```

OBSERVATION:

This program fetches weather information for one or more cities using the OpenWeatherMap API. It shows each city's temperature, humidity, and weather description in a clean format. Every time it runs, it also saves the results into a local JSON file, adding each new entry to the list. The user can enter cities and API keys through

command-line arguments or by typing them when prompted. Overall, it provides an easy way to look up weather data and keep a growing record of all queries.